Computation for Analytics

DIANE WOODBRIDGE, PH.D

Motivation

At the end of the Day2, you should be able to write a script to automatically synch your local Git repository with the remote Git repository.

Contents

Version Control

- Git

Programming

Python

- Python Development
- Python Libraries

Debugging

Error/Exception Handling

Contents

Version Control

- Git

Programming

Python

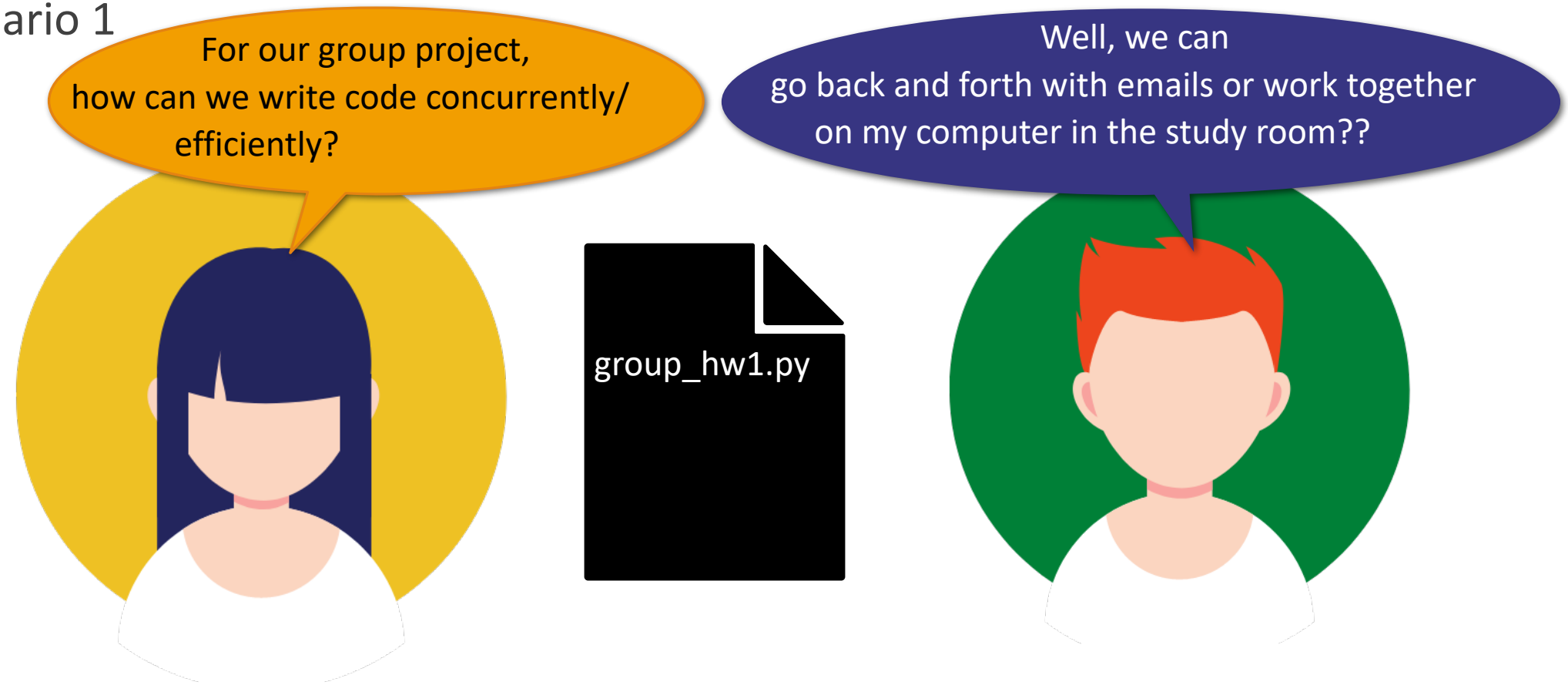
- Python Development
- Python Libraries

Debugging

Error/Exception Handling

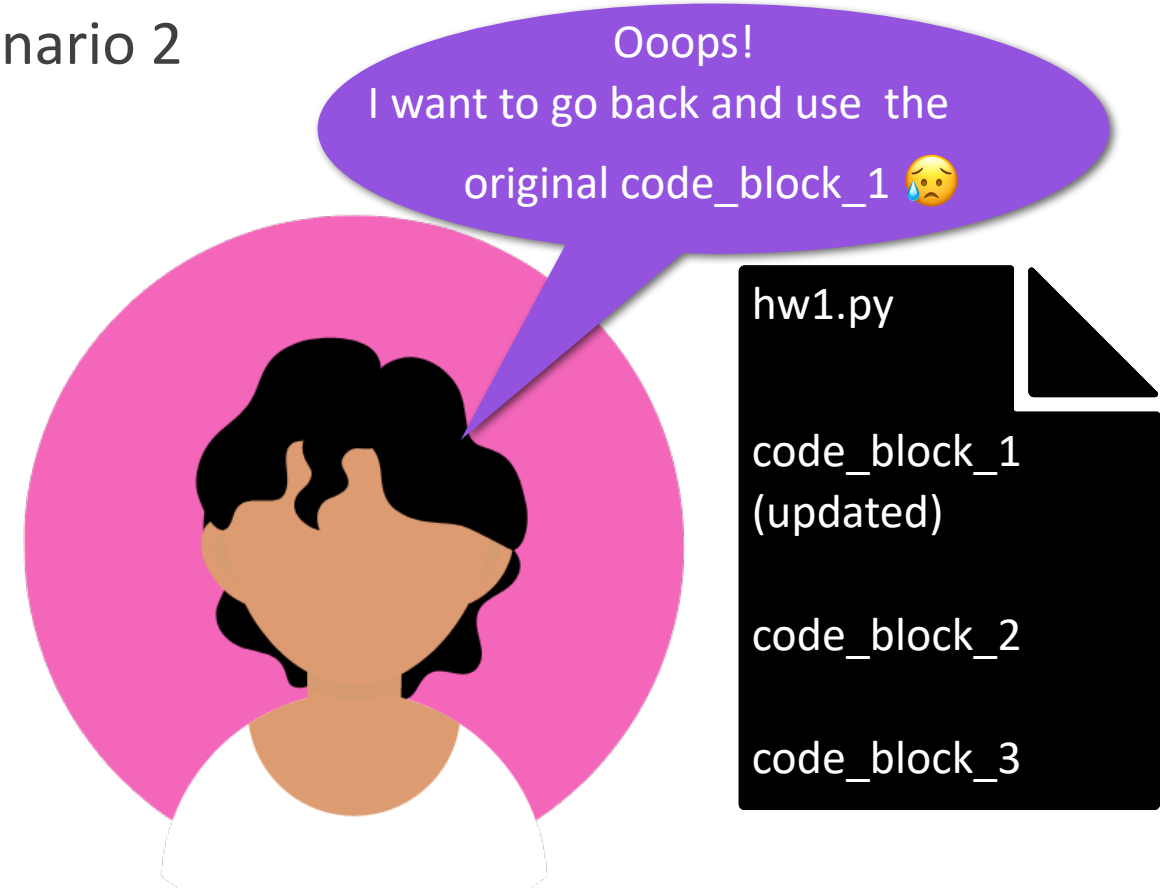
Version Control System (VCS)

Scenario 1



Version Control System (VCS)

Scenario 2



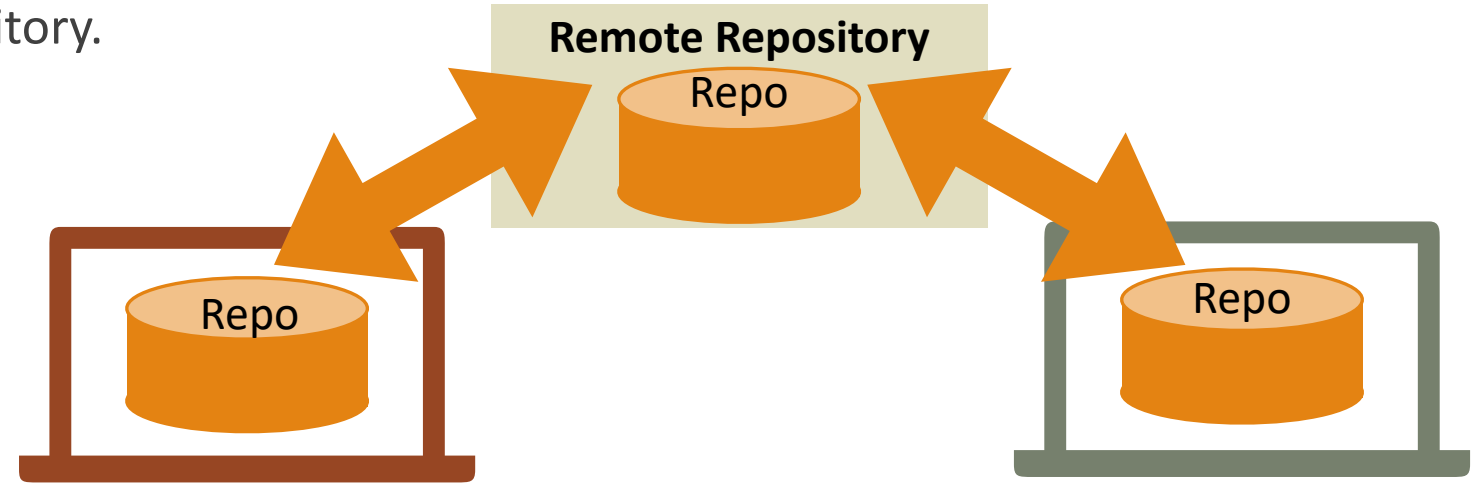
Version Control System (VCS)

System tracking and managing changes to software code

- Allows multiple people to operate on different copies of the repository without getting confused or losing changes. **Great for collaboration and backing up files and their changes.**
- Repository : Directory containing files and directories.

Distributed VCS (DVCS)

- Each client has a local database that tracks changes and stores the copies to the remote central repository.

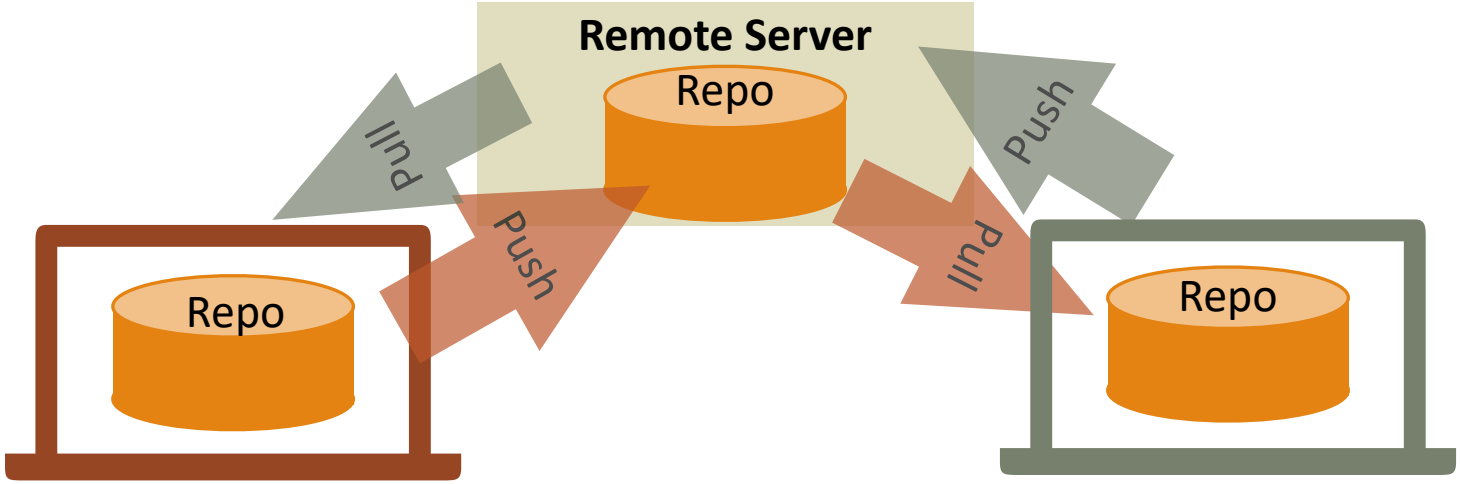


https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control

Git

DVCS that tracks changes to files and directories within a repository

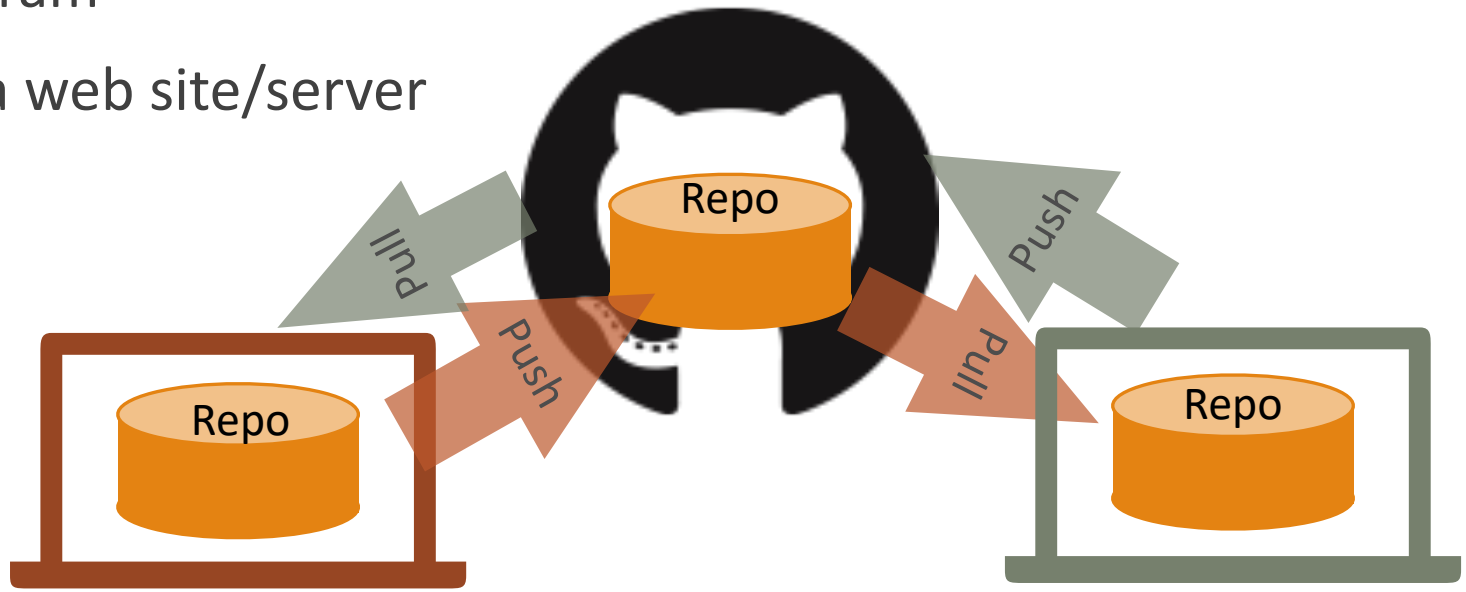
- Workers push/pull changes from a repo on one machine to a repo on a collaborator's machine
- Git is a program runs on your local machines and remote server.



Git vs Github

github.com

- A remote server that hosts repositories, making collaboration much easier
- Note: git != github.com
  - git is program
  - github is a web site/server



Git

Motivation

- Every developer uses it, as it enhances efficiency and collaboration capacities at work or for open-source projects.
- Every company uses it

Git

First time set up

- Installation
  - `$conda install -c anaconda git`

- Configuration - stored in ~/.gitconfig
  - `$git config --global user.name "user_name"`
  - `$git config --global user.email "user_email"`

Git

If you want to see available git commands and their details,

- `$git --help [command]`

```
start a working area (see also: git help tutorial)
   clone             Clone a repository into a new directory
   init              Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
   add               Add file contents to the index
   mv                Move or rename a file, a directory, or a symlink
   restore           Restore working tree files
   rm                Remove files from the working tree and from the index
   sparse-checkout   Initialize and modify the sparse-checkout

examine the history and state (see also: git help revisions)
   bisect            Use binary search to find the commit that introduced a bug
   diff              Show changes between commits, commit and working tree, etc
   grep              Print lines matching a pattern
   log               Show commit logs
   show              Show various types of objects
   status            Show the working tree status

grow, mark and tweak your common history
   branch            List, create, or delete branches
   commit            Record changes to the repository
   merge             Join two or more development histories together
   rebase            Reapply commits on top of another base tip
   reset             Reset current HEAD to the specified state
   switch            Switch branches
   tag               Create, list, delete or verify a tag object signed with GPG

collaborate (see also: git help workflows)
   fetch             Download objects and refs from another repository
   pull              Fetch from and integrate with another repository or a local branch
   push              Update remote refs along with associated objects
```

Git

First time set up

- For our classes, we will use GitHub
  - Create an account at https://github.com/signup

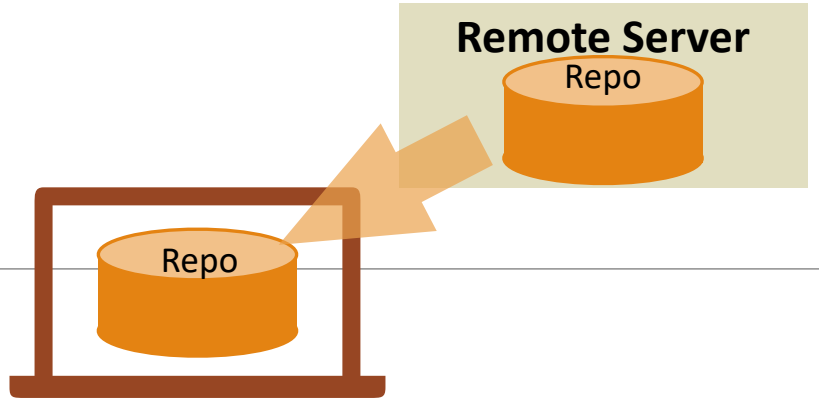
Git

Repository (Repo)

- The directory that we are tracking is called a **repository (repo)**.
  - It includes a hidden subdirectory called **.git** including a database of all changes
  - You will create a separate repo for different projects. Ex. A separate repo for EDA.
- All files/folders to be tracked by the repo should be located under that directory.
- We need to tell git **when** and **which** files to to take a snapshot in the repo directory.

Git

Create repo



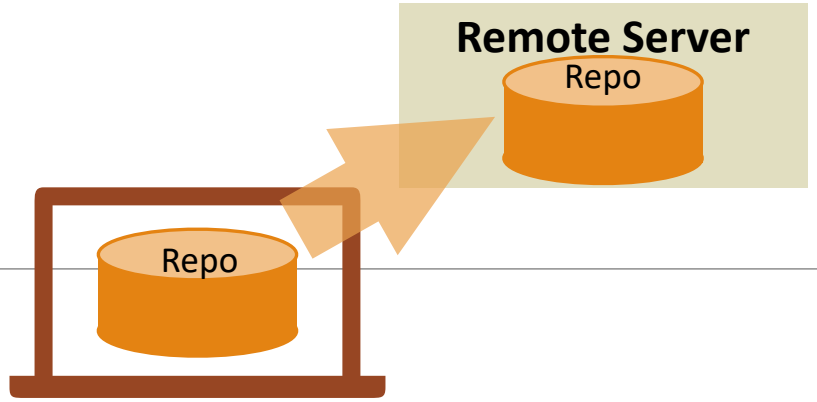
1. Using an existing GitHub repository
   - `$git clone repository_url path_to_clone_repo`
     - If you don't provide path_to_clone_repo, it will clone to your current working directory with the same name in the url
     - If it asks access tokens, follow this instruction.

```
ML-ITS-901885:2022_MSDS501 dwoodbridge$ git clone https://github.com/dianewoodbridge/2019-msds694-example ../2019_MSDS694/2019-msds694-example_new
Cloning into '../2019_MSDS694/2019-msds694-example_new'...
remote: Enumerating objects: 110, done.
remote: Counting objects: 100% (15/15), done.
remote: Compressing objects: 100% (6/6), done.
remote: Total 110 (delta 11), reused 9 (delta 9), pack-reused 95
Receiving objects: 100% (110/110), 5.07 MiB | 12.95 MiB/s, done.
Resolving deltas: 100% (58/58), done.
```

https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens

Git

Create repo

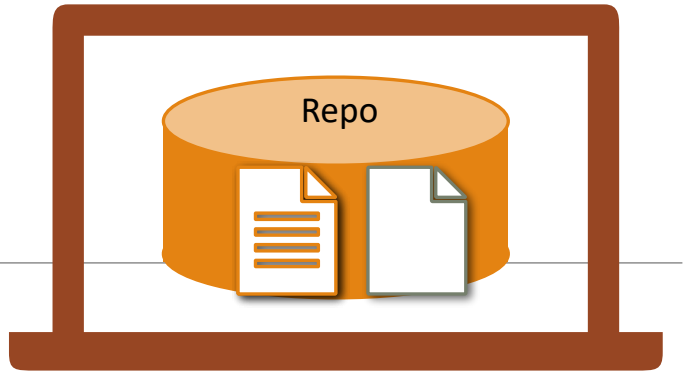


2. Creating a new GitHub repository
   - First create a repo (https://github.com/new)
   - Be in the directory to track files
   - `$git init`
   - `$git remote add origin` *`url`*

```
ML-ITS-901885:2022_MSDS501 dwoodbridge$ mkdir 2022_msds501_example
ML-ITS-901885:2022_MSDS501 dwoodbridge$ cd 2022_msds501_example/
ML-ITS-901885:2022_msds501_example dwoodbridge$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /Users/dwoodbridge/Class/2022_MSDS501/2022_msds501_example/.git/
ML-ITS-901885:2022_msds501_example dwoodbridge$ git remote add origin https://github.com/dianewoodbridge/msds501_computation_2022
```

Git

Add/update/check files to be tracked in the repo



1. Define files to be tracked and add changes

   ```
   $git add file_name
   $git add * (: If you want to track everything)
   ```

2. Check files being tracked and modified (added, deleted, renamed, etc.).

   ```
   $git status
   ```

```
ML-ITS-901885:2022_msds501_example dwoodbridge$ git status
On branch master

No commits yet

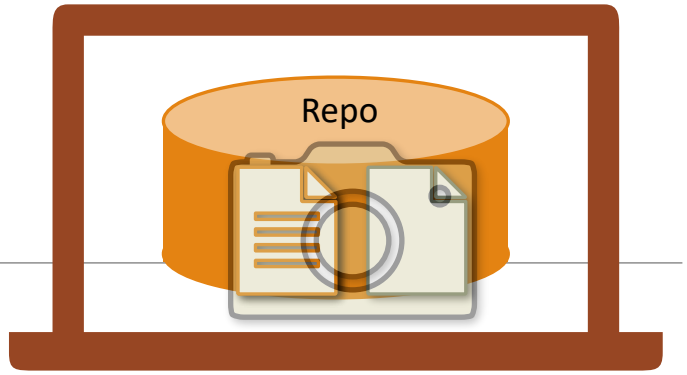
Untracked files:
  (use "git add <file>..." to include in what will be committed)
        Day2/

nothing added to commit but untracked files present (use "git add" to track)
ML-ITS-901885:2022_msds501_example dwoodbridge$ git add Day2
ML-ITS-901885:2022_msds501_example dwoodbridge$ git status
On branch master

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   Day2/ex01_git_commands.sh
```

Git

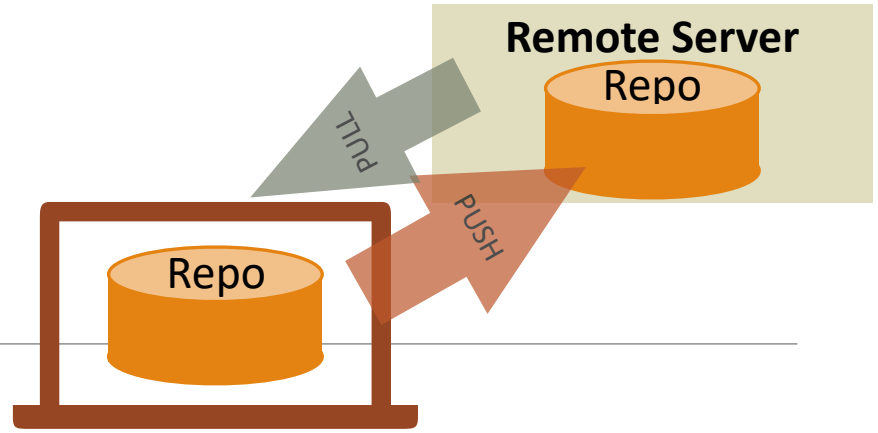


Commit Changes

- git tracks snapshots called a *commit* and each commit includes the series of updates that you made by `git add, remove`, etc. as a chunk.
  `$git commit –a –m "messages"`
  - `–m` option : commit with messages
  - –a option : automatically commit files that have been modified/deleted

```
ML-ITS-901885:2022_msds501_example dwoodbridge$ git commit -a -m "added ex01"
[master (root-commit) 89d46fc] added ex01
 1 file changed, 8 insertions(+)
 create mode 100644 Day2/ex01_git_commands.sh
```

Git



Working with remote repo

- Pull - Fetching changes and merging with the current branch.
  - `$git pull [remote] [branch]`
    - By default, *remote*'s name is origin, and branch's name is master
- Push - Sharing the commits
  - `$git push [remote] [branch]`
  - Every push ensures that the complete file set and git change database (in .git subdirectory) is mirrored at GitHub.

```
ML-ITS-901885:2022_msds501_example dwoodbridge$ git pull origin master
From https://github.com/dianewoodbridge/msds501_computation_2022
 * branch            master     -> FETCH_HEAD
Already up to date.
```

```
ML-ITS-901885:2022_msds501_example dwoodbridge$ git push origin master
Enumerating objects: 4, done.
Counting objects: 100% (4/4), done.
Delta compression using up to 16 threads
Compressing objects: 100% (2/2), done.
Writing objects: 100% (4/4), 375 bytes | 187.00 KiB/s, done.
Total 4 (delta 0), reused 0 (delta 0), pack-reused 0
remote:
remote: Create a pull request for 'master' on GitHub by visiting:
remote:      https://github.com/dianewoodbridge/msds501_computation_2022/
remote:
To https://github.com/dianewoodbridge/msds501_computation_2022
 * [new branch]      master -> master
```

- Note : Always pull the changes, resolve conflicts where developers might worked on together but has different updates, and then push

Git

Commands Summary

	Commands	Explanation
Configuration	$git config --global user.name "user_name" $git config --global user.email "user_email"	Add default user name and email to be used in git
Creating Repo	$git clone repository_url path_to_clone_repo	Create, using an existing GitHub repository
	$git init $git remote add origin url	Create an empty Git repository Connect with a remote repo
Checking Status	$git status	Show the state of the current directory and files
Defining Files to Track	$git add file_name	Defining files to track changes
Committing Changes	$git commit -a --m "messages"	Stage eiles
Pulling Changes	$git pull [remote] [branch]	Fetch changes and merge with the current branch
Pushing Changes	$git push [remote] [branch]	Sharing the commits

- We will cover other git topics including branch as the program proceeds.

Git

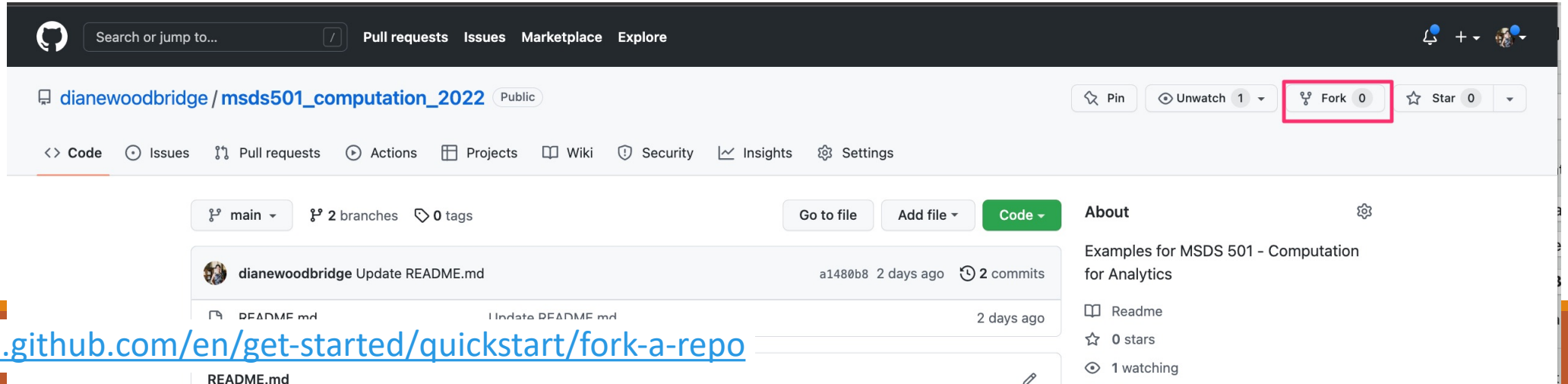
Some other useful commands

Command	Explanation
$git rm filename	Remove a file from the directory and from git repo tracking
$git mv from_filename to_filename	Rename a file or directory managed by git
$git log	Shows the commit logs including its Secure Hash Algorithm (SHA), author, date, and message
$git reset --hard SHA/HEAD	Wipe out any changes you've made to managed files, resetting the repository to the previous commit (SHA)/ the most recent commit(HEAD)

GitHub

GitHub Fork

- This is not a Git command and just provided by GitHub.
- Create a connected copy of the repository and make changes without affecting the upstream repository.
  - This is for a repo that you don't have a write access or you don't want to write/update.
  - Fork creates a copy to your GitHub and your push is not going to affect to the original repo.
- Create Fork => Clone that to your local repo (instead of the original url)

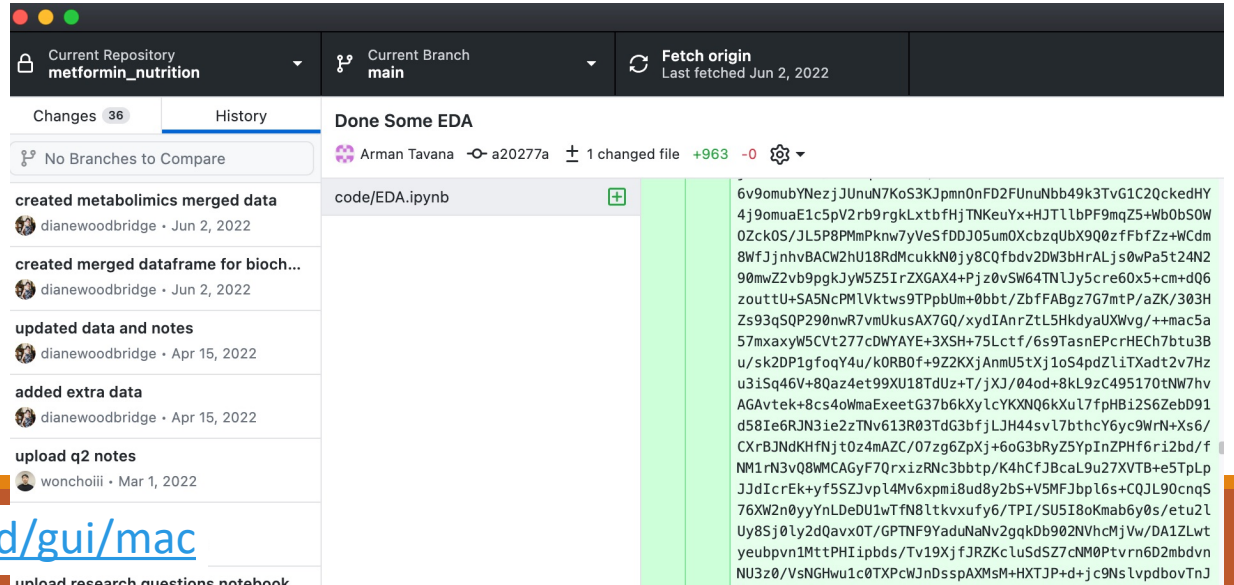


https://docs.github.com/en/get-started/quickstart/fork-a-repo

GitHub

GUI Tools

- I found that it is easy to start with GUI to familiarize with Git and see the differences/updates when you're working with multiple people.
  - If you're using GitHub, you can use GitHub Desktop
  - In general, Source Tree and other GUI tools are also useful.



https://git-scm.com/download/gui/mac

Contents

Version Control

- Git

Programming

Python

- Python Development
- Python Libraries

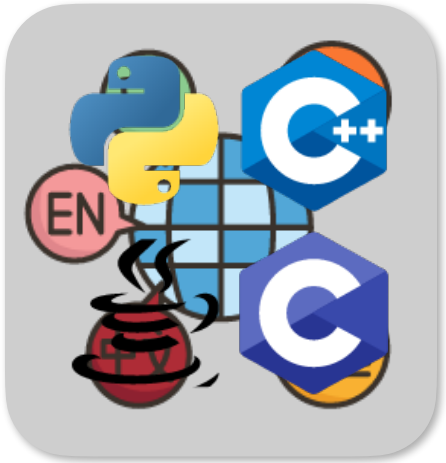
Debugging

Error/Exception Handling

Programming

- A process of designing and implementing computer programs
- As computer cannot understand human languages directly, we're using programming languages including Python, C++, JAVA, etc. that the machine can interpret and understand.
- Programming is a defining a logic to solve problems using a programming language.





Contents

Version Control

- Git

Programming

Python

- Python Development
- Python Libraries

Debugging

Error/Exception Handling

Python 

A high-level, general-purpose and the most popular Scripting language

- The language that data scientists use

You will use four interfaces to access a Python interpreter:

- pythontutor.com (no setup and visualizes Python executions)
- The UNIX CLI to run Python .py files and to interact with OS
- Jupyter notebooks, a browser interface for code and notes
- Visual Studio Code (VS Code), an application for programming
- We'll even learn to execute code on a remote cloud computer like Amazon Web Services (AWS), Google Cloud Platform (GCP)

https://www.python.org/

Python

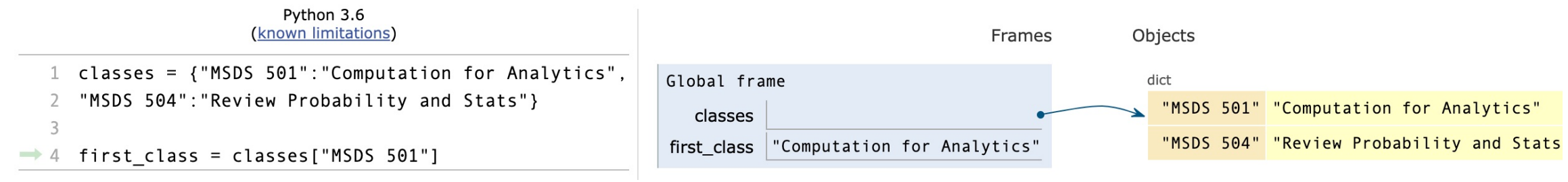
.ipynb and .py file extensions

- .ipynb
  - iPython Notebook file extension (has its own format)
  - You can run this on Jupyter notebook / lab / VSCode
- .py
  - Regular python extension (plain text that you can open on text editors)
  - You can execute it on your terminal `$python your_code.py`

Python Interface

pythontutor.com

- Edit and execute code without installing software on your laptop
- Visualizes the state of the Python interpreter
- Really nice interactive tool for learning Python and other languages



Python Interface

Command Line

- The command line is built in to Macs / UNIX machines
- Python interpreter installed with Anaconda
  - For interactive mode, you can run using `$python` (But this won't be saved)
    - Interactive mode prints your intermediate outputs, but this is for your convenience.
  - For .py file you can run using `$python file_name.py`

```
(DistributedComputing) ML-ITS-901885:2022_msds501_example dwoodbridge$ python
Python 3.7.11 (default, Jul 27 2021, 07:03:16)
[Clang 10.0.0 ] :: Anaconda, Inc. on darwin
Type "help", "copyright", "credits" or "license" for more information.
>>> classes = {"MSDS 501":"Computation for Analytics",
"MSDS 504":"Review Probability and Stats"}
>>> classes["MSDS 504"]
'Review Probability and Stats'
(DistributedComputing) ML-ITS-901885:Day2 dwoodbridge$ python ex02.py
```

Python Interface

Jupyter Notebook/Lab

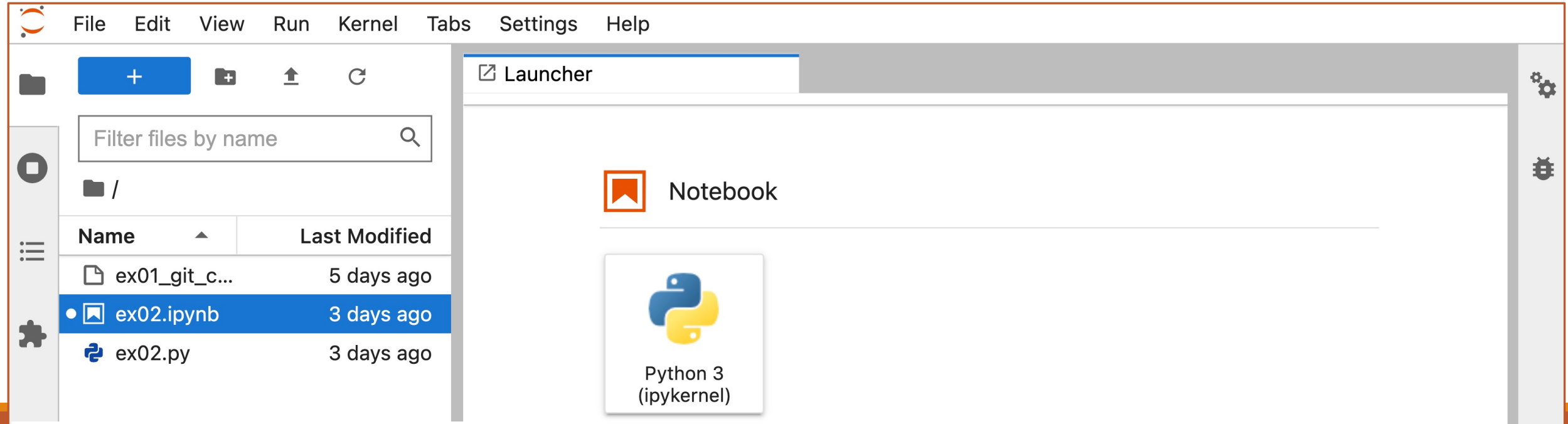
- Interactive execution for code, displaying its execution outcome.
  - Installed automatically with Anaconda
  - To start,
    `$jupyter lab`

```
(base) ML-ITS-901885:Day2 dwoodbridge$ jupyter lab
Note: NumExpr detected 16 cores but "NUMEXPR_MAX_THREADS" not set, so enforcing safe limit of 8.
NumExpr defaulting to 8 threads.
[I 2022-06-30 06:39:56.489 ServerApp] jupyterlab | extension was successfully linked.
[I 2022-06-30 06:39:56.529 ServerApp] Writing Jupyter server cookie secret to /Users/dwoodbridge/Library/Jupyter/runtime/jupyter_cookie_secret
[W 2022-06-30 06:39:59.679 ServerApp] The module 'jupyter_nbextensions_configurator' could not be found. Are you sure the extension is installed?
[I 2022-06-30 06:39:59.679 ServerApp] nbclassic | extension was successfully linked.
[I 2022-06-30 06:39:59.679 ServerApp] panel.io.jupyter_server_extension | extension was successfully linked.
[I 2022-06-30 06:40:00.009 ServerApp] nbclassic | extension was successfully loaded.
[I 2022-06-30 06:40:00.011 LabApp] JupyterLab extension loaded from /Users/dwoodbridge/opt/anaconda3/lib/python3.7/site-packages/jupyterlab
[I 2022-06-30 06:40:00.011 LabApp] JupyterLab application directory is /Users/dwoodbridge/opt/anaconda3/share/jupyter/lab
[I 2022-06-30 06:40:00.017 ServerApp] jupyterlab | extension was successfully loaded.
```

Python Interface

Jupyter Notebook/Lab

- Interactive execution for code, displaying its execution outcome.
- You can use keyboard shortcuts to edit/run.



https://jupyterlab.readthedocs.io/en/stable/

Python Interface

Visual Studio Code (VS Code)

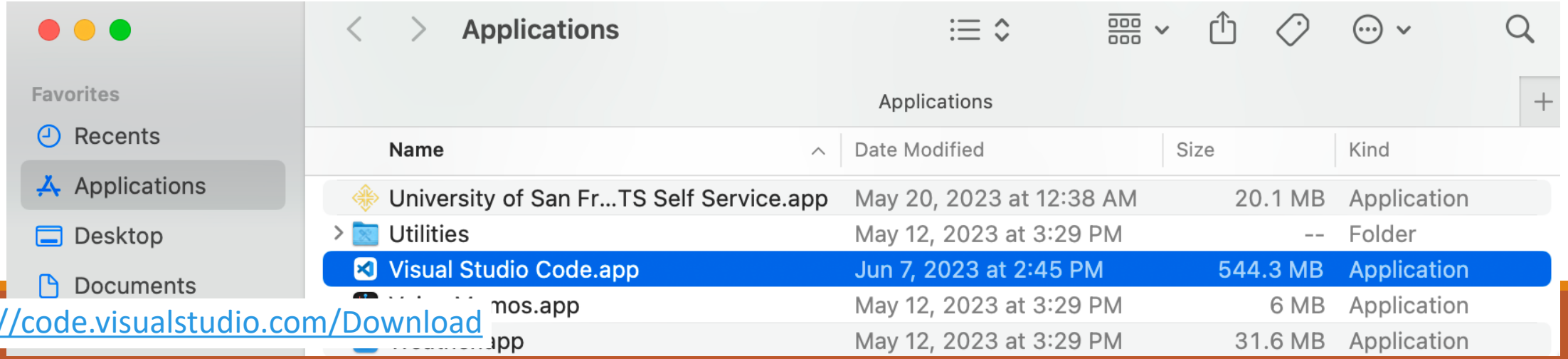
- VS Code is an Integrated Development Environment (IDE).
- VS Code has an extensive debugging capabilities and supports various languages including C++, etc.
- With its Python extension, you can use VS Code for writing codes, debugging, choosing virtual environments, etc.
  - You can run both .ipynb,.py and terminal commands using VS Code easily without launching Jupyter lab.

https://code.visualstudio.com/docs/python/python-tutorial

Python Interface

Visual Studio Code (VS Code)

- Installation
  - Go to the Visual Studio Code Download page and download and install.
  - Move it to Applications folder
  - Add /Applications/Visual Studio Code.app/Contents/Resources/app/bin to the environment variable, PATH

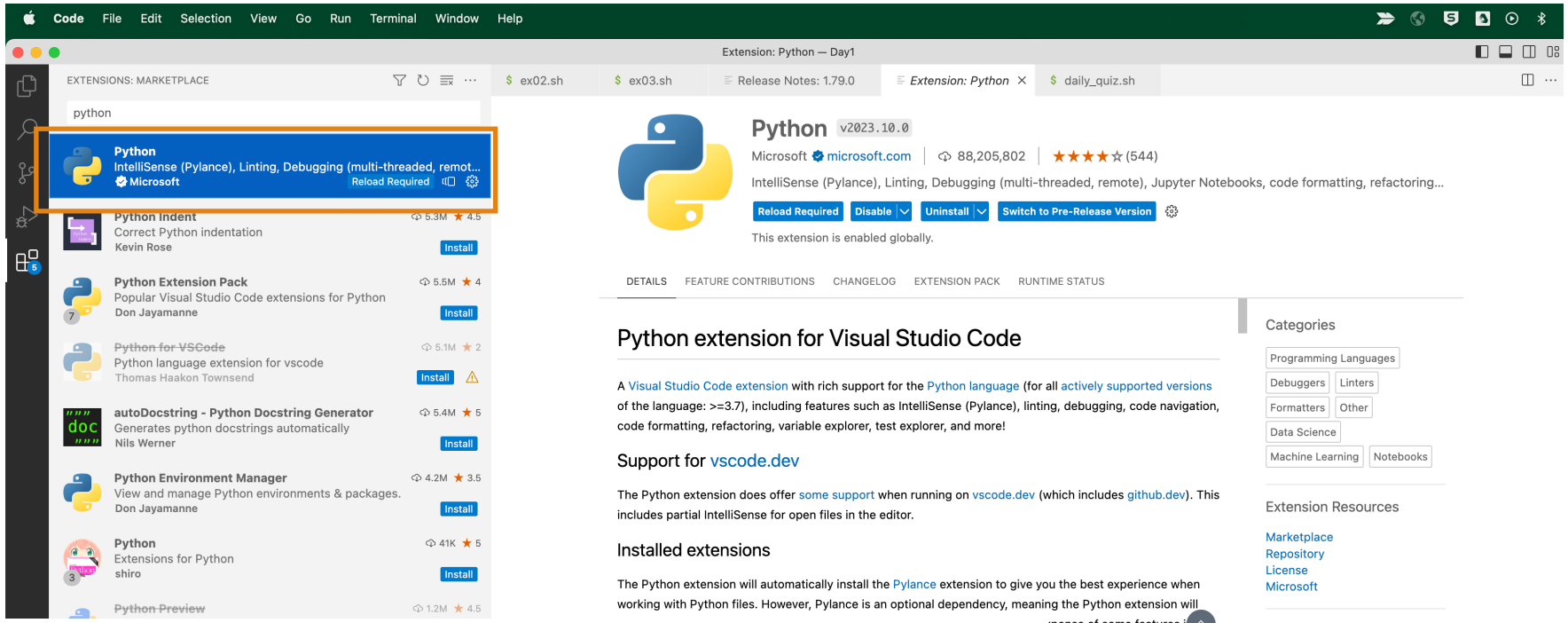


https://code.visualstudio.com/Download

Python Interface

Visual Studio Code (VS Code)

- Add Python Extension



https://code.visualstudio.com/docs/python/python-tutorial
https://marketplace.visualstudio.com/items?itemName=ms-python.python

Python Module/Packages/Libraries

Module

- A simple Python file (.py)

Packages

- A simple directory having collections of modules.
- It includes __init__.py as a constructor.

Libraries

- A collection of related functionality of codes / Collection of packages
- You can install it by `$conda install library_name` and import it using `import library_name`

The terms, packages and libraries are often used interchangeably.

Python Libraries/Packages

Useful Libraries and Packages for Data Science

- NumPy : High-level mathematical functions
- SciPy : Scientific computing and technical computing
- Pandas : Data manipulation and analysis
- SciKit-Learn : Machine learning library
- Keras : Artificial neural networks
- TensorFlow : Machine learning and artificial intelligence
- PyTorch : Neural networks
- Matplotlib : Plotting library
- Plotly : Plotting library
- Scrapy : Web-crawling framework
- BeautifulSoup : Parsing HTML and XML

Contents

Version Control

- Git

Programming

Python

- Python Development
- Python Libraries

Debugging

Error/Exception Handling

Debugging

Process of finding causes of bugs/errors and fixing them.

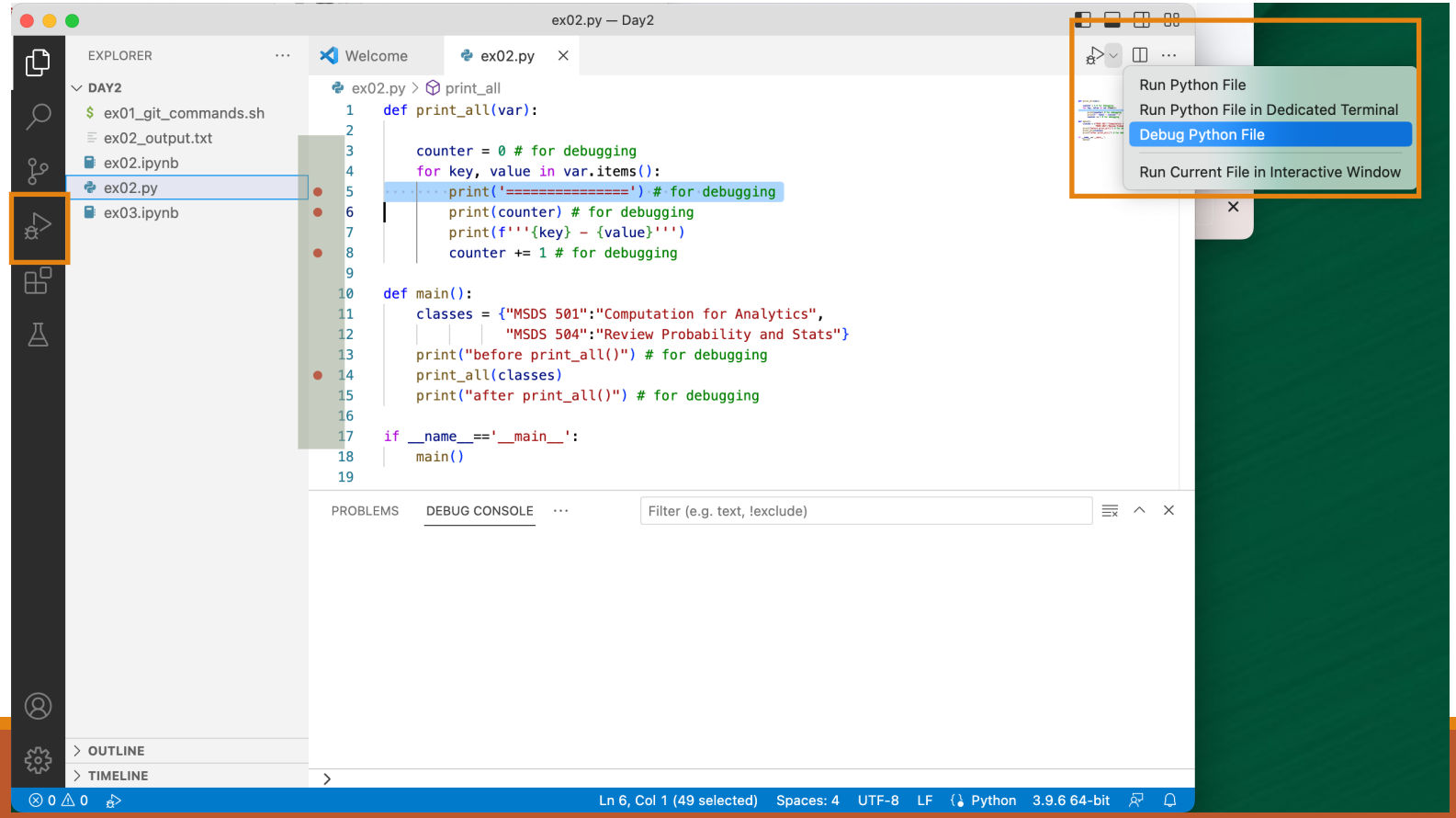
- Important to read the error messages - Those could be detailed enough that you can google to find high-level solutions.
  - You might still want to see where it is being caused and how to fix it by "debugging" your code.
    - You can 1) use a debugging tool on your IDE or 2) print statements to see the status/value could be helpful.

https://jupyterlab.readthedocs.io/en/stable/user/debugger.html
https://code.visualstudio.com/docs/python/debugging

Debugging

You can toggle a breakpoint to pause in order to inspect your codes.

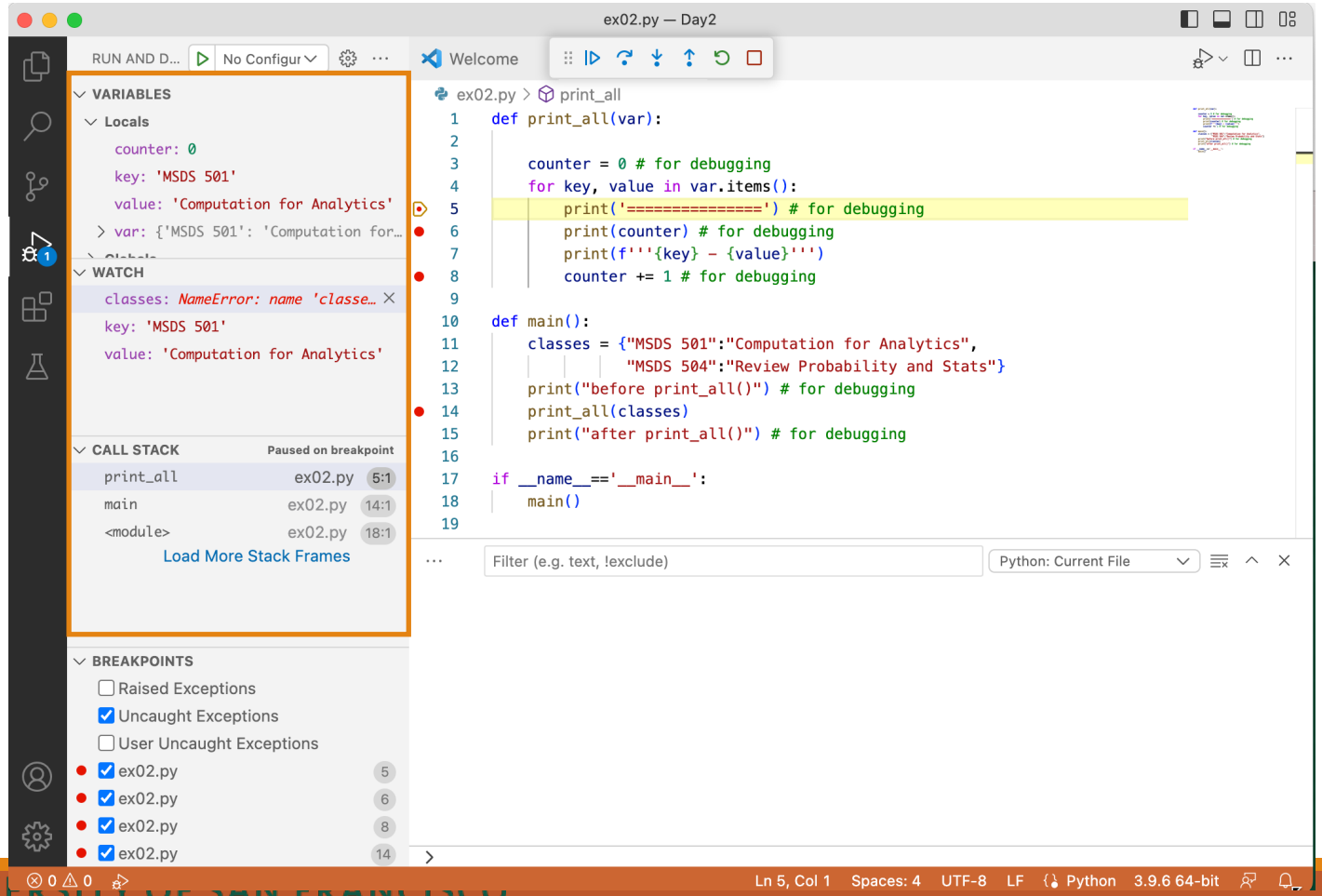
Start run a debugger (choose Python) to debug your code.



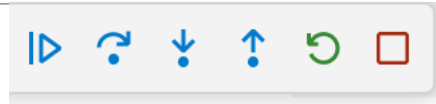
Debugging

You can inspect your variables

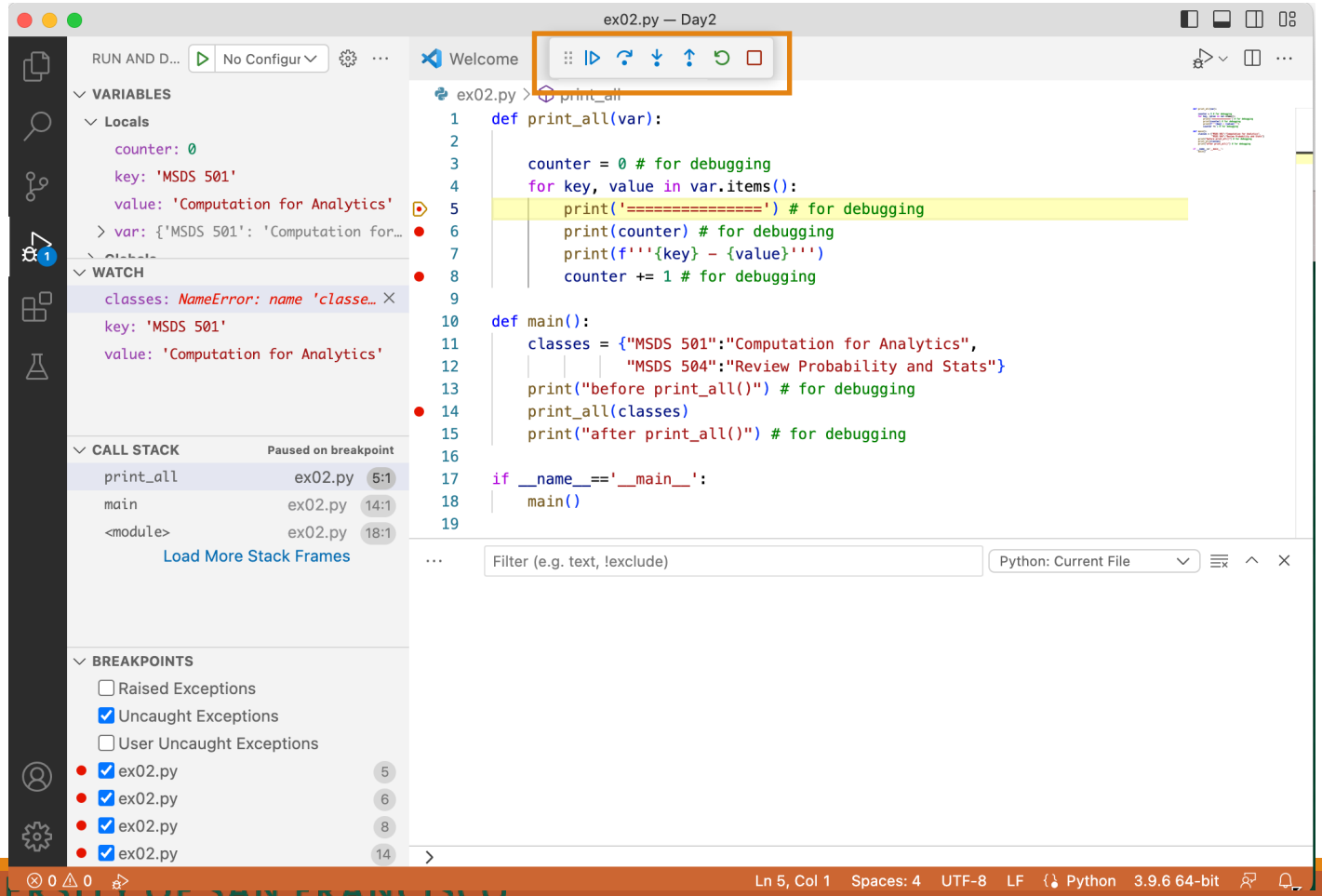
- CALL STACK - subroutines of your code
- VARIABLES - variables of your current call stack
- WATCH - You can define a variables to be monitored



Debugging

Debugging Actions 

- Continue/Pause - Continue/pause the execution until the next breakpoint (F5)
- Step Over - Execute the next methods as a single command without inspecting (F10)
- Step Into - Enter the next method and execute line by line (F11)
- Step Out - Exit the subroutine and return to the call stack (Shift + F11)
- Restart - Restart the debugging process (Shift + Control + F5)
- Stop (Shift + F5)



https://code.visualstudio.com/docs/editor/debugging

Example 2

Using print statement, track your variables and values.

```
(base) ML-ITS-901885:Day2 dwoodbridge$ python ex02.py
before print_all()
===============
0
MSDS 501 - Computation for Analytics
===============
1
MSDS 504 - Review Probability and Stats
after print_all()
```

```
(base) ML-ITS-901885:Day2 dwoodbridge$ python ex02.py  > ex02_output.txt
```

Contents

Version Control

- Git

Programming

Python

- Python Development
- Python Libraries

Debugging

Error/Exception Handling

Error/Exception Handling

For some cases, Python will catch serious conditions and throws **Exceptions,** while most errors cannot be detected until runtime.

- Some exception types - See which line causes this issue.
  - SyntaxError : Fix wrong syntax
  - IndentationError : Fix incorrect indentation
  - TabError: Fix inconsistent uses of tabs and spaces.
  - TypeError: an operation or function is applied to an object of inappropriate type
  - ArithmeticError : arithmetic errors including ZeroDivisions
  - ModuleNotFoundError : importing a library that is not installed. Need to fix with conda install
  - NameError : Raised when a local or global name is not found. Make sure the name is defined

Best way to fix it??

Copy and paste to Google.
Google and Stackoverflow are now your best friends.

https://docs.python.org/3/library/exceptions.html

Example

```
import pytorch
```

```
---------------------------------------------------------------------------
ModuleNotFoundError                       Traceback (most recent call last)
Input In [2], in <cell line: 1>()
----> 1 import pytorch

ModuleNotFoundError: No module named 'pytorch'
```

Example



Google

ModuleNotFoundError: No module named 'pytorch'

All Images Videos News Shopping More Tools

About 650,000 results (0.40 seconds)

https://stackoverflow.com › questions › no-module-na...

No module named "Torch" - python - Stack Overflow

Feb 23, 2019 · 20 answers

Try to install **PyTorch** using pip: First create a Conda environment using: conda create -n env_pytorch python=3.6.

"**no module named** torch". But installed **pytorch** 1.3.0 with ...	Feb 7, 2020
ModuleNotFoundError: **No module named** 'torch'	Nov 27, 2019
ImportError: **No module named** torch - Stack Overflow	Aug 5, 2020
Conda - **ModuleNotFoundError**: **No module named** 'torch'	Mar 10, 2021

More results from stackoverflow.com

Error/Exception Handling

You can handle exceptions using `try, except` statements.

- Syntax

```
try:
    code_that_may_cause_exceptions
except exception_error_type_1:
    error_handling_code_1
except exception_error_type_2:
    error_handling_code_2
```

Example 3

The following codes have errors.

- inputs : supposed to have only integers, but may contain strings
- When exception happens, print an message indicating the index number of wrong data types considering the error types.

[1]:
```
inputs = [1, 2, 3, 4, "9", "10"]
```

[2]:
```
print(sum(inputs))
```

Contents

Version Control

- Git

Programming

Python

- Python Development
- Python Libraries

Debugging

Error/Exception Handling

References

Git : https://git-scm.com/book/en/

GitHub : https://docs.github.com/en/

Jupyter: https://www.python.org/

Jupyter Lab : https://jupyterlab.readthedocs.io/en/stable/

Python : https://docs.python.org/3/library/exceptions.html

Visual Studio Code : https://code.visualstudio.com/docs/python